

实验二：关于 secp256k1 仓库

一、实验目标

check repo <https://github.com/bitcoin-core/secp256k1>, 撰写报告分析其中与比特币密码算法使用相关的漏洞修复和性能改进, 说明这些修改背后的原因及数学原理。

二、相关原理

2.1 关于 secp256k1 仓库

secp256k1 仓库是一个用于数字签名及其他加密原语的高性能高保证 C 库, 适用于 secp256k1 椭圆曲线上的其他密码学原语。该库旨在成为 secp256k1 曲线上公开的最高质量密码学库。然而, 其开发的主要重点是比特币系统中的使用, 与比特币不同, 使用方式可能测试不足、验证不足, 或者界面设计不够完善(这一条看起来是免责声明)。

这个库的特色有:

1. secp256k1 ECDSA 签名/验证及密钥生成。
2. 秘密/公钥的加法和乘法调整。
3. 串译/解析密钥、公钥、签名。
4. 恒定时间、恒定内存访问签名和公钥生成。
5. 非随机化 ECDSA (通过 RFC6979 或呼叫者提供的函数)。
6. 非常高效的实现。
7. 适用于嵌入式系统。
8. 没有运行时依赖。
9. 公钥恢复的可选模块。
10. ECDH 密钥交换可选模块。
11. 根据 BIP-340, Schnorr 签名可选模块。

12. 根据 BIP-324, ElligatorSwift 密钥交换的可选模块。
13. 根据 BIP-327, MuSig2 Schnorr 多重签名可选模块。
14. 根据 BIP-352, 用于无声支付发送和接收的可选模块。

实现细节:

1. 概述

- (1) 没有运行时堆分配。
- (2) 完善的测试基础设施。
- (3) 结构设计便于审查和分析。
- (4) 旨在移植到任何支持 C89 编译器和 `uint64_t` 的系统。
- (5) 不用 floating 类型。
- (6) 只暴露高级接口, 以最小化 API 表面并提升应用安全性。(“想不安全地使用都难”)

2. 有限域上的运算 (核心运算)

以曲线场大小 ($2^{256} - 0x1000003D1$) 模算术的优化实现。

- (1) 使用 5 个 52 位 limbs
- (2) 使用 10 个 26 位 limbs (包括 Wladimir J. van der Laan 为 32 位 ARM 手工优化的组装)。这是一个实验性功能, 尚未获得足够的审查以达到本库的质量标准, 但已开放供社区测试和评审。

3. 标量运算 (私钥相关)

优化实现时, 没有数据依赖的算术分支模曲线阶数。

- (1) 使用 4 个 64 位 limbs (依赖编译器的 `__int128` 支持)。使
- (2) 用 8 个 32 位 limbs。

4. 基于 `safegcd` 并做了一些修改的模逆 (包括场元和标量), 以及可变时间变体 (由 Peter Dettman 设计)。

5. 群运算

- (1) 针对曲线方程的点加法公式 ($y^2 = x^3 + 7$)。

- (2) 在雅可比坐标和仿射坐标中，尽可能使用点间加法。
 - (3) 必要时使用统一的加加/加倍公式，以避免依赖数据的分支。
 - (4) 在雅可比坐标空间中，通过比较进行点/x 的比较，但没有场反演。
6. 验证签名时的乘法 ($aP + bG$)。
- (1) 点乘数使用 wNAF 符号。
 - (2) 对于 G 的倍数，使用更大的窗口，使用预先计算好的倍数。
 - (3) 用沙米尔的技巧同时用公钥和生成器进行乘法。
 - (4) 利用 secp256k1 的高效计算自同态，将 P 倍数分解成两个半大小的倍数。
7. 签名时的乘法
- (1) 使用预先计算好的 16 次方乘以生成元的表格，使一般乘法变成一系列加法。
 - (2) 旨在完全避免用于密钥操作的时序侧通道（在合理的硬件/工具链上）
 - ① 通过无分支的条件变换访问表，使内存访问保持一致。
 - ② 无数据依赖分支
 - (3) 可选的运行时盲化，以对抗差分分析。
 - (4) 预计算的表会添加并最终减去那些已知标量（秘密密钥）未知的点，甚至防止对密钥有控制权的攻击者，从而在内部控制数据。

三、分析

secp256k1 是一个仍在使用并不断更新的库，截至最终写完本实验报告前它的 commits 中已经标到#1901 了，短时间内完全学习并掌握并找到符合条件的某次修改并理解其原理还是相对困难的。从总体上讲，secp256k1 这个比特币核心使用的加密库是一个为比特币打造的、极其硬核的密码学库，它在速度和安全性之间做了极致平衡，所有复杂设计（多种位数表示、各种优化技巧）最终都是为了让私钥在运算时绝不泄露任何信息，同时保持跨平台的高性能。

在本次报告中，为了寻找到一个适合分析与比特币密码算法使用相关的漏洞修复或性能改进的代码修改，我们选择在 commits 页面中翻来翻去尝试找到合适条目。

结果令人失望，在比较新的条目中进行的修改通常不涉及算法本身，比如被编号为#1886 的这个：移除弃用的 secp256k1_context_no_precomp 指针，在这次修改中移除了弃用的指针，所以理论上讲这个或许确实算是算漏洞修复的范围（大概吧？虽然弃用这个指针本身就是计划中的一部分（因为在上一个修改中才取消了对它的使用）所以好像不太算是漏洞），但和预期中要分析的算法改动还是区别很大的。

于是进一步尝试，使用页内搜索搜索关键词“fix”以期望寻找到可以被称得上“修复”的更新。

结果依旧，或许是因为这个库本身就在不断地高频小幅更新，所以在最近的百十条 commits 中依旧没发现十分典型的案例，但找到了例如：“修复了一个 GCC17snapshot 警告（#1881）”“修复了一个 dead check（#1867）”这样的条目。相比于最初所想象的“重大算法优化”或者“重大漏洞修复”来讲似乎并没有想象中那么夸张（也有可能那样的夸张的更新在更早的 commits 中才能找到）。

选择适时退而求其次，以#1867 为例，我们来分析一下这个漏洞修复：

原代码：

```
secp256k1_ge_is_infinity(&aggnonce_pt[i]);
```

修改后代码：

```
CHECK(secp256k1_ge_is_infinity(&aggnonce_pt[i]) == 1);
```

修改描述：由于缺少“CHECK”，返回值被丢弃，这里实际上没有检查。

额，好直白的漏洞修复。再努力一下吧，再找一条看看：

“Merge #1837: tests: Fix function pointer initialization C89 error in ellswift tests”：修复了 ellswift 测试中 C89 函数的初始化错误，这个

“pedantic compliance error”（额，这咋翻译，迂腐规范错误？）出现时会
导致函数指针的初始化不被允许。在本次代码中修改了
`secp256k1_ellswift_xdh_hash_function` 指针的初始化形式。